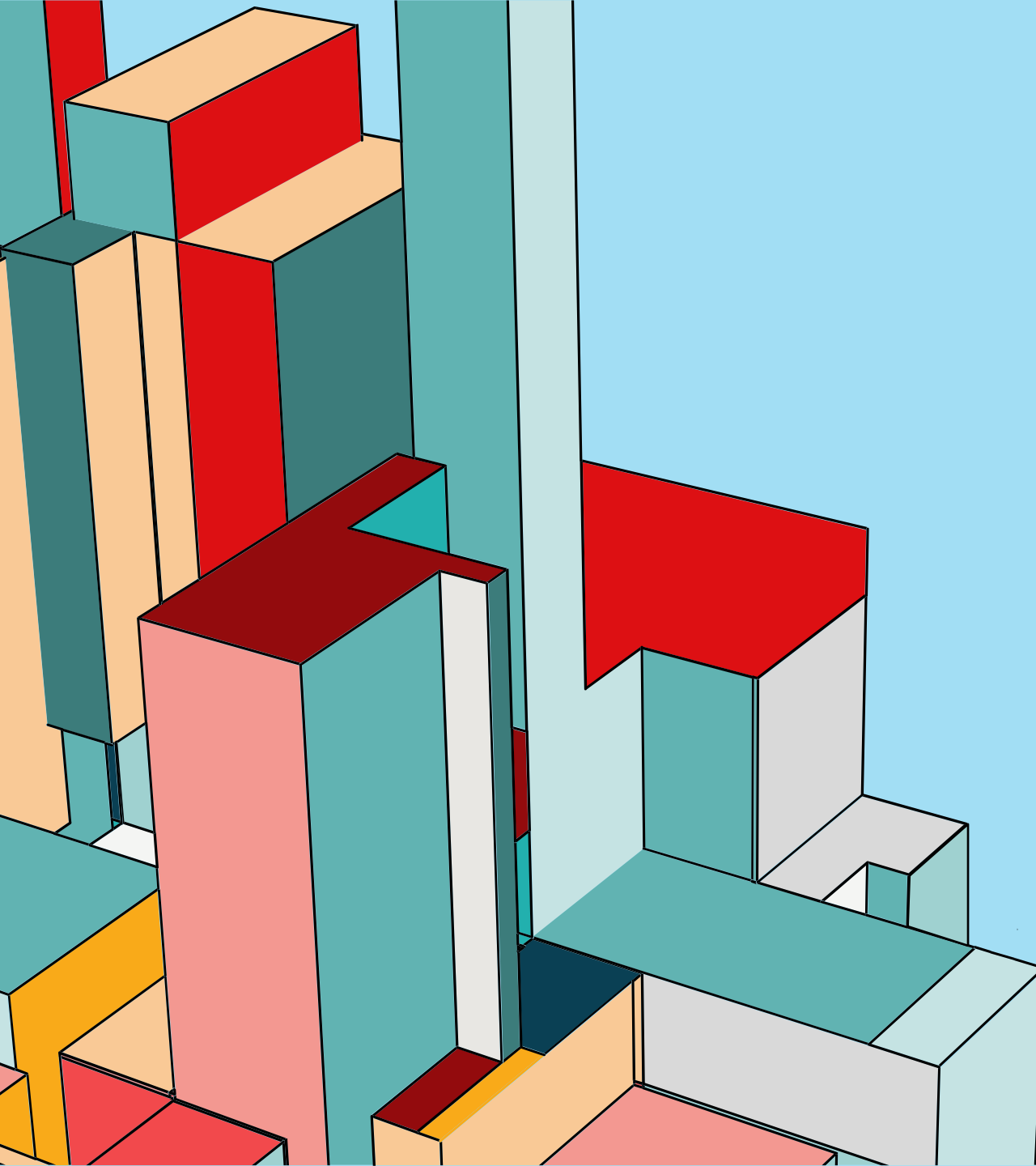


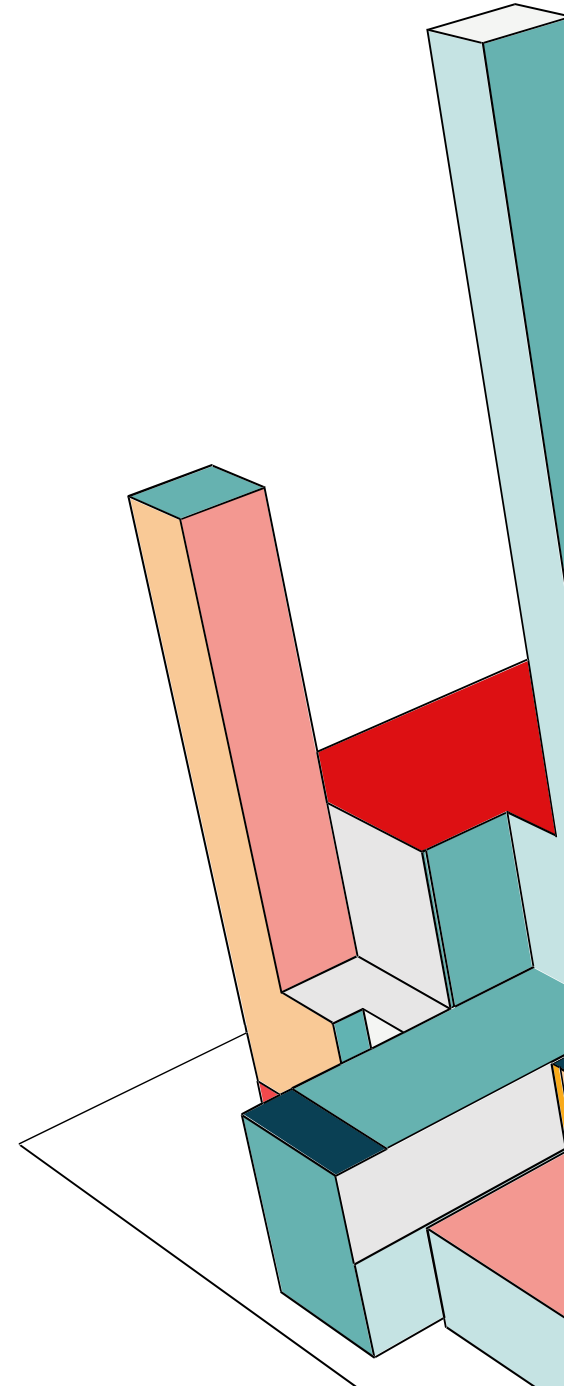


LECTURE 14 **HASHING II**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

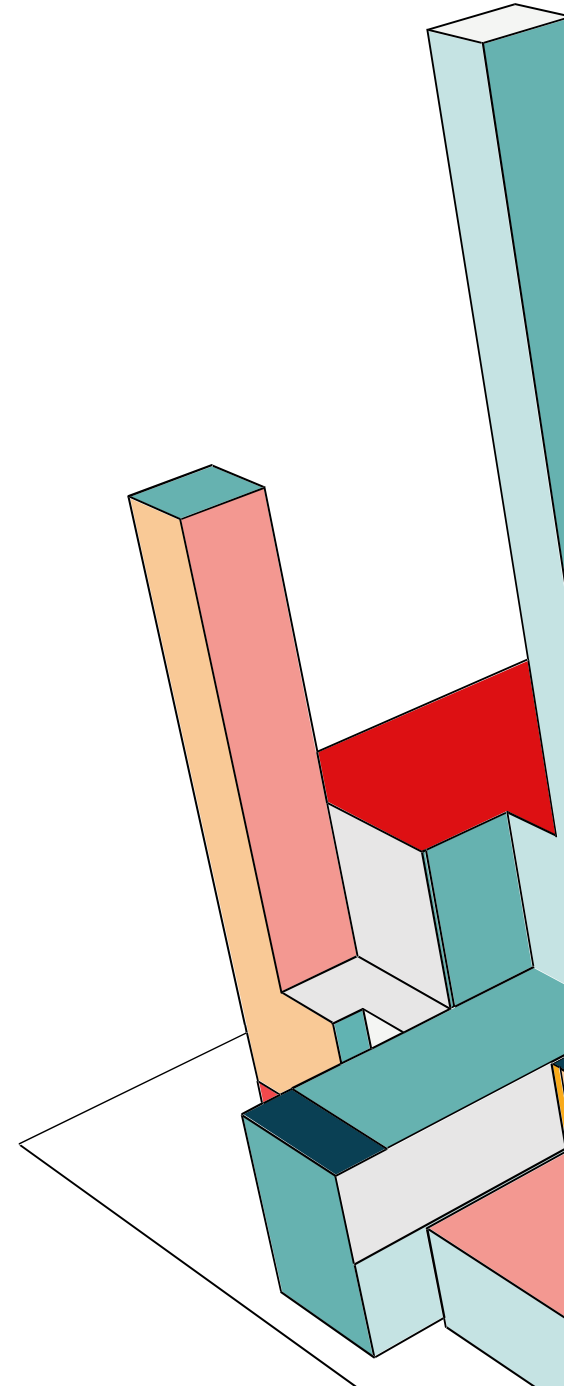
ANNOUNCEMENTS & REMINDERS

- Talk this evening: "50 Years of AI" by Prof. Hendler, 7:30pm in CBIS Auditorium
- Exam 2 next week on Wednesday (3/18)
- Quick discussion of lab task 3



TODAY'S TOPICS

- Closed hashing
 - Linear & quadratic probing
 - Double hashing
- Applications where collisions are desirable
 - Bloom filters
 - Locality sensitive hashing (LSH)

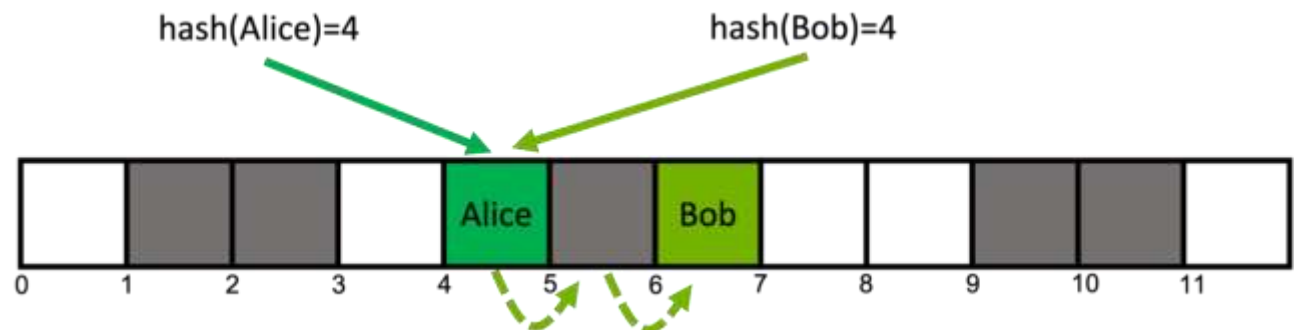


CLOSED HASHING

Some applications (e.g. caching) require implementing a hash table in hardware – actual circuitry. Obviously, an open hashing design like chaining won't work – you can't just ask the OS for more circuitry to store more keys in one bucket.

Instead, these systems implement *closed hashing* (irritatingly also called *open addressing*): When a collision occurs, the item being inserted into the table is placed in a different (empty) bucket.

Note that closed hashing cannot accommodate $n > m$ – once we're full, we can't add anything else without deleting something first.



PROBING

How do we know which bucket to use next after a collision?
And what happens if that bucket is also full?

These questions are answered by the *probing* scheme used.

All probing schemes amount to adding a probe function to the primary hash function. This new function takes an input $i = 0, 1, 2, \dots$ representing how many tries we've made so far to find an empty bucket.

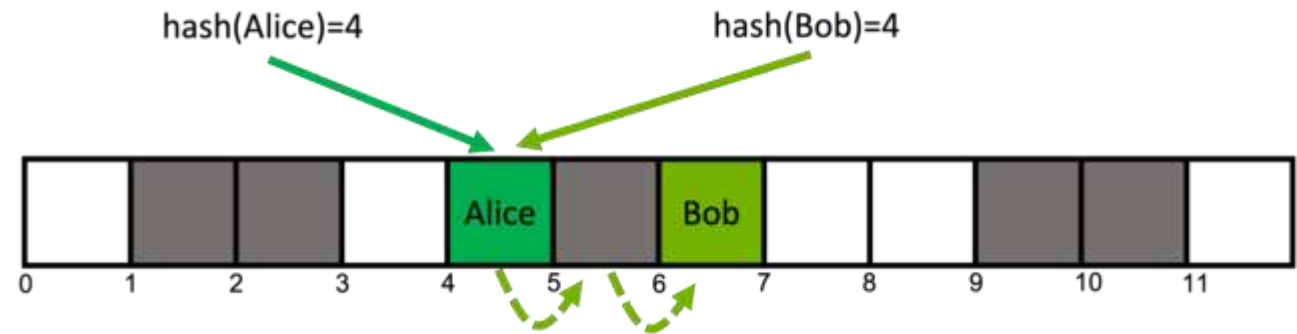
$$h(k, i) = (h'(k) + p(k, i)) \bmod m$$

As you might expect, there are many possible types of probe function that we might use.

LINEAR PROBING

The simplest scheme is *linear probing* – just try the next bucket down the table!

$$h(k, i) = (h'(k) + i) \bmod m$$



One can also replace the i term with ci – skipping ahead a fixed number of buckets at each step.

QUADRATIC PROBING

The downside of linear probing is *clustering* – once a region of the table starts to need probing, that area of the table tends to accrete more and more entries.

Quadratic probing attempts to alleviate this by making the probing steps continually increase in size.

$$h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$$

SUBMITTY POLL #1

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

$$h'(k) = k \bmod m$$

Let $m = 11, c_1 = 3, c_2 = 2$.

We hash the values 17, 20, 34, 2, 28.

In which bucket does 28 end up?



DOUBLE HASHING

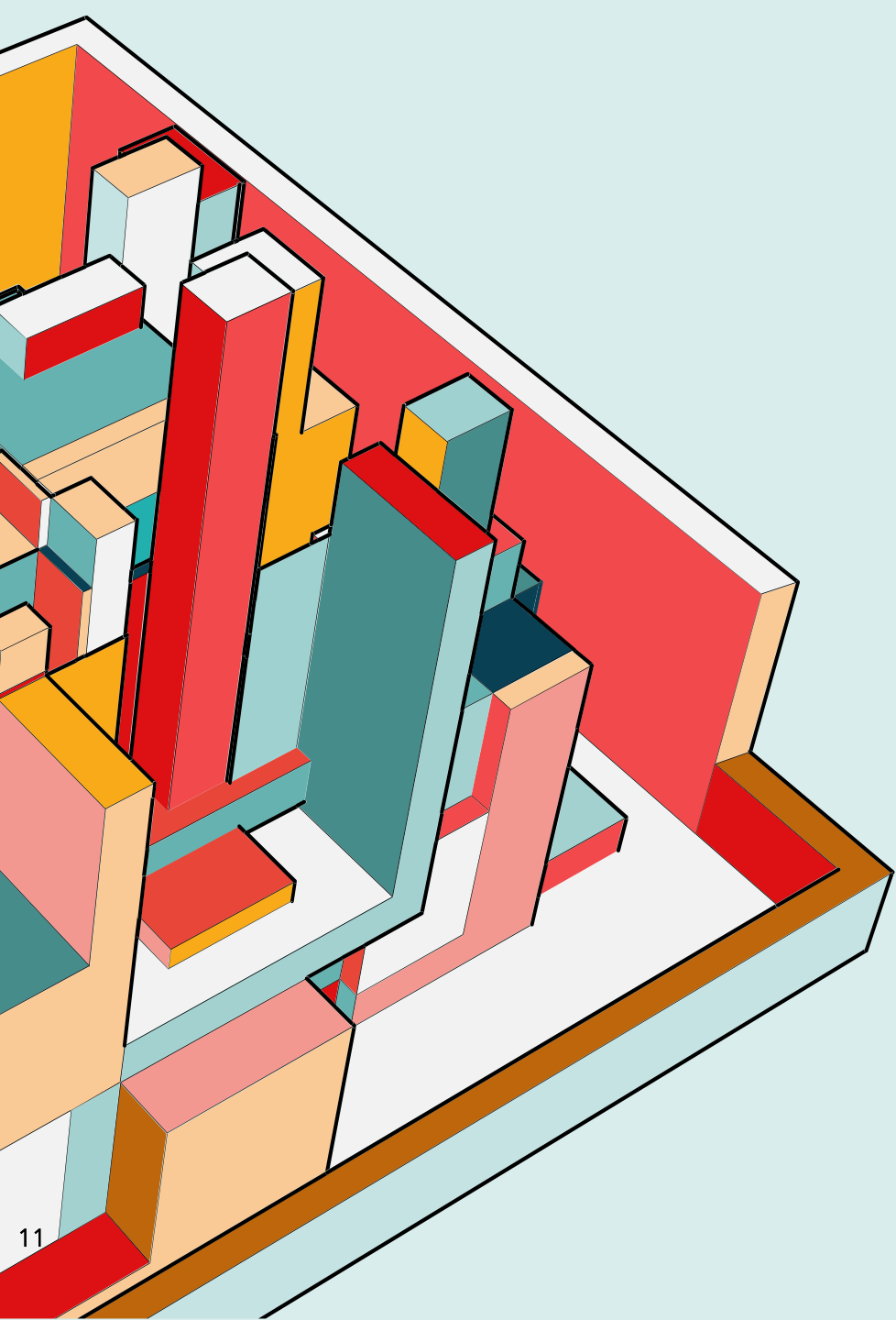
Quadratic hashing still produces some secondary clustering. *Double hashing* is a little slower, but comes pretty close to a uniform selection from the remaining options at every probe step. It uses two hash functions:

$$h(k, i) = (h'_1(k) + ih'_2(k)) \bmod m$$

ANALYSIS OF CLOSED HASHING

Bottom line: If we assume simple uniform hashing, the expected number of probes required for an unsuccessful search (and thus also an insertion) are $1/(1 - \alpha)$.

For a successful search, the expected value is $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.



BLOOM FILTERS

A PROBLEM IN WEB CACHING

Consider the task of caching web pages in your browser – that is, storing them locally so that when the user requests a page again, it can be displayed without a (slow) web retrieval.

By default, you would cache each object requested. But: ~75% of all web page accesses are “one-hit wonders” – pages that the user looks at once and never again. Caching these pages is a waste of resources – a direct use of more storage space, plus the larger cache makes it slower to find things in it.

Solution: Remember which pages you have looked at once, and only cache them the second time a user requests the page. We need these checks to be fast, though...

A MODIFIED DICTIONARY STRUCTURE

A regular hash table handles the question “Have we seen this before?” quite well...

... but what if storage space is very limited? A hash table requires $O(m + n)$ space – the entire premise is that we get good speed by using more space to store the data.

Observe that we don’t need all of the dictionary methods here: We’re only using Insert and Search, but not Delete. How might we take advantage of that – ideally in a way that limits the amount of storage space needed?

BLOOM FILTERS

It turns out that if we are willing to tolerate the occasional wrong answer, we can drastically reduce the amount of space needed to answer the question "Have we seen this before?"

A *Bloom filter* is a probabilistic summary data structure that will occasionally produce a false positive, but never a false negative.

When it says you have seen something before, it might be wrong.

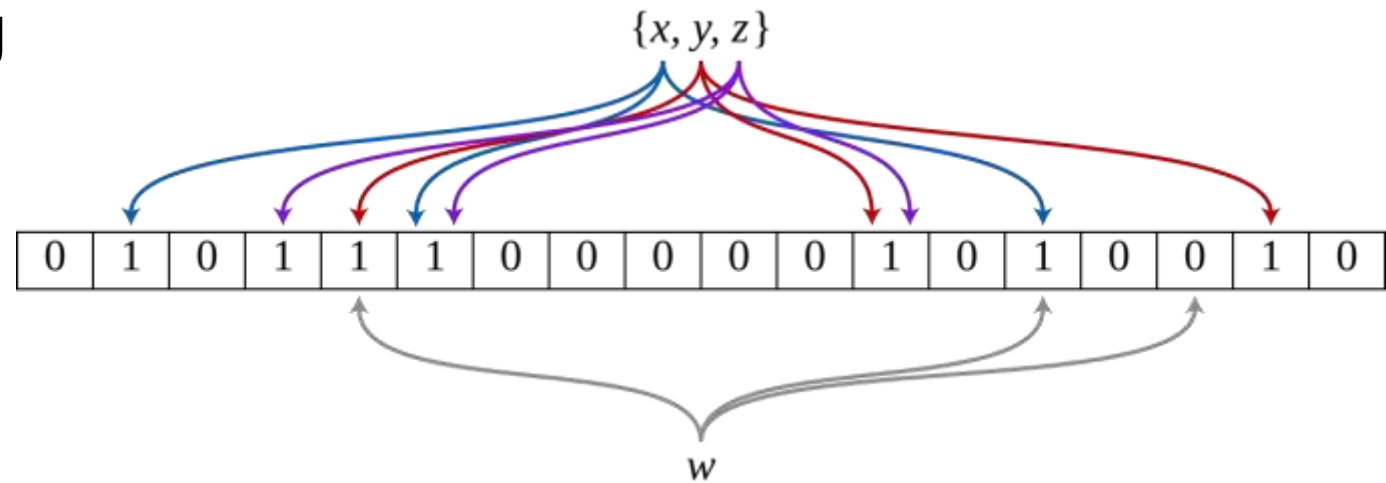
When it says you haven't seen something before, it's always correct.

The really striking thing about Bloom filters is how small they can be: To produce a false positive probability of 1%, a Bloom filter needs less than 10 bits of space per element – regardless of the size of the elements!

BLOOM FILTERS – HOW IT WORKS

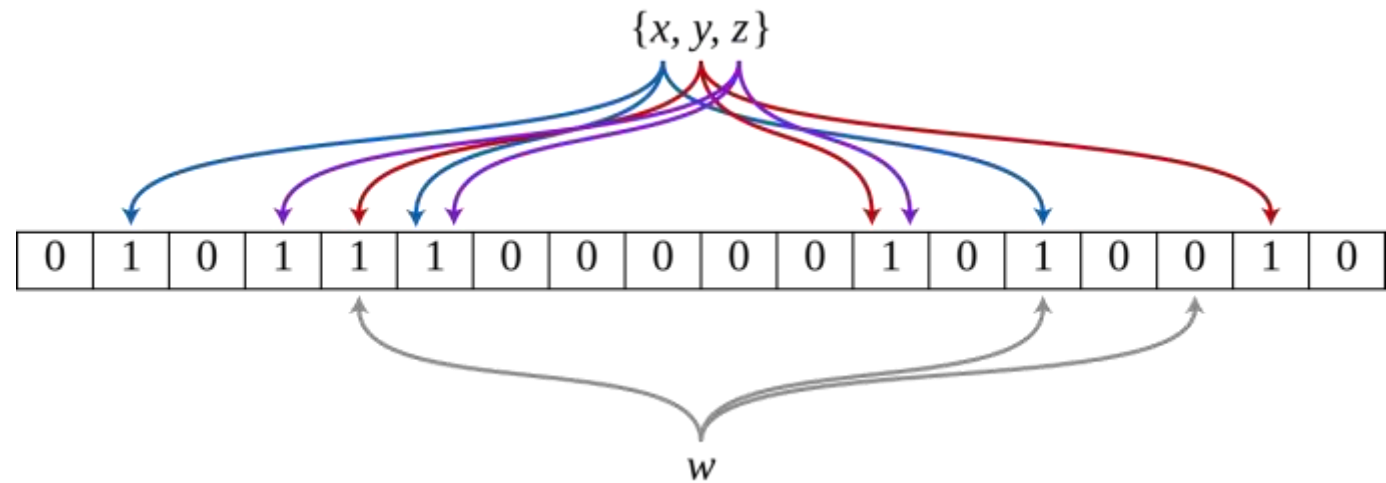
A Bloom filter is an m -bit array and a set of k different hash functions that map keys onto the range $[0, m - 1]$. For best results, the hash functions should be independent. The array is initialized to all 0s.

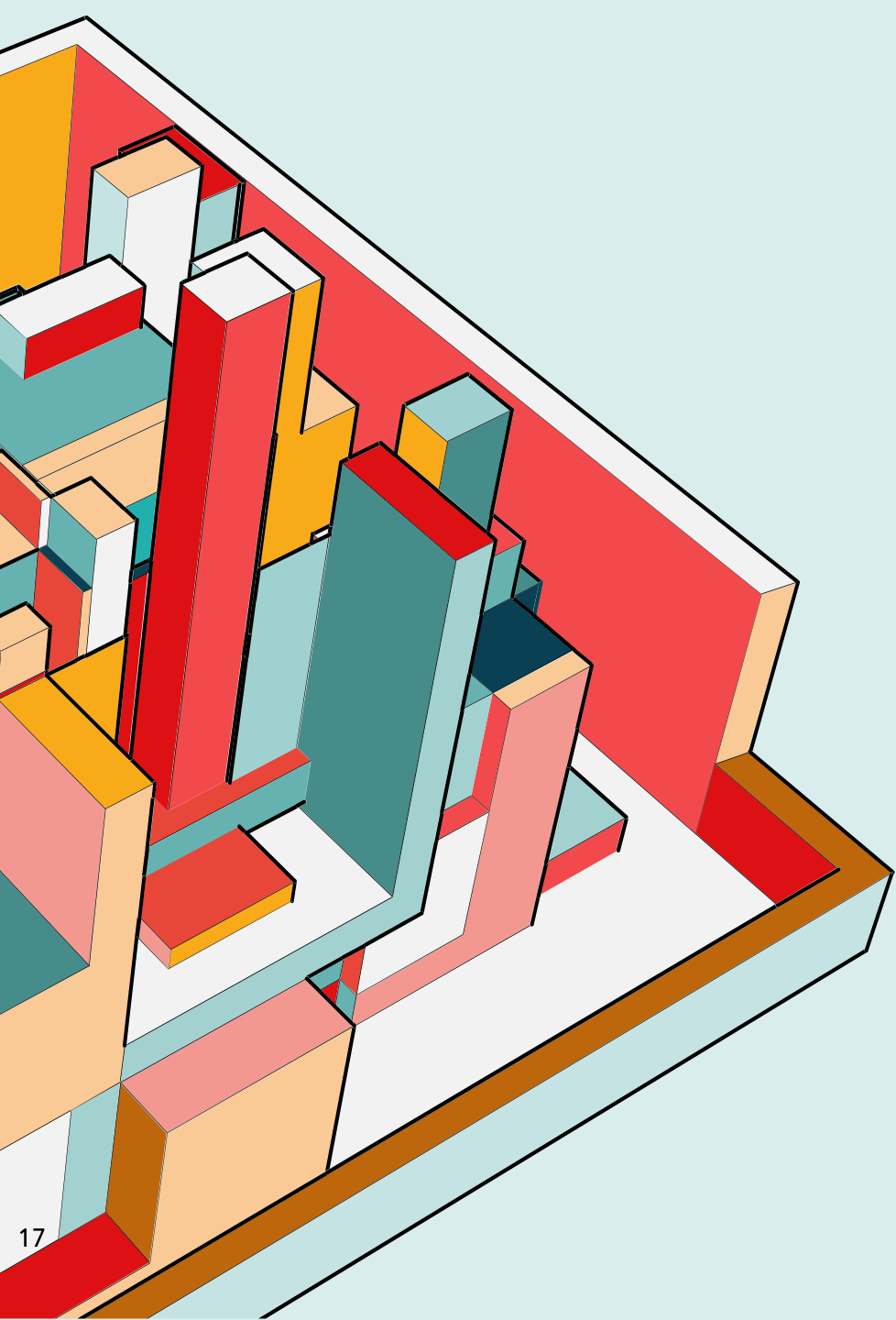
To add a key to the Bloom filter, we simply hash the key using each of the hash functions and flip the resulting bits from 0 to 1.



BLOOM FILTERS – HOW IT WORKS

To check if a value has been seen before, we do the same thing, except that if any of the bits are 1, we return “probably yes”. If all of the bits are 0, we return “definitely not”.





LOCALITY-SENSITIVE HASHING (LSH)

COLLABORATIVE FILTERING

One area where a LOT of computer science research efforts have gone in the last 20 years is *recommender systems*.

The general problem is this: Given a collection of user-media associations (really, this is just a bipartite graph), possibly with weights or other descriptions, determine which users are similar to each other (and which media are like each other).

For a few years in the late '00s, NetFlix ran an [open competition](#) seeking recommender algorithms that beat their own in-house system. The \$1M grand prize was claimed after three years.



SIMILARITY OF GAMES

For a concrete example, let's stick with games. In the simplest case, a profile for each game is simply a list of users that own/play/like the game.

Given just this information, how might we measure the similarity between two games?

$$\begin{aligned} p_1 &= 0 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ \dots \\ p_2 &= 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ \dots \end{aligned}$$

In this case, a 0 doesn't provide much information – just because someone hasn't played a game doesn't automatically mean they don't like it. The 1s are where it's at.

JACCARD SIMILARITY

$p_1 = 0 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ \dots$

$p_2 = 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ \dots$

A common way to measure the similarity of two sets is the *Jaccard similarity*. It has the advantage of being very easy to calculate:

$$J(p_1, p_2) = |p_1 \cap p_2| \div |p_1 \cup p_2|$$

SUBMITTY POLL #2

Let $A = \{a, b, d, g, i, j\}$ and $B = \{b, c, d, e, f, h, j\}$. What is the Jaccard similarity of these two sets?



USING OUR SIMILARITY METRIC

Now that we have a similarity metric, how do we actually determine which games are similar to each other?

Observe that the naive approach of just testing each game against each other game is an $O(n^2)$ operation. If n is big (it is), this becomes problematic.

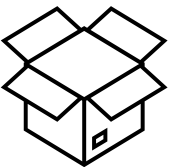
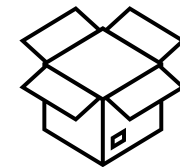
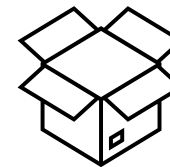
Additionally: We probably don't need the specific score for every pair of games. All we really want is a collection of games that are similar enough.

By this point, hopefully it has occurred to you that hashing may provide a solution to our problem!

A SIMILARITY-PRESERVING HASH FUNCTION

The key insight is that in a standard hash table, collisions are a bad thing. For this problem, collisions are a good thing – what we would like to do is hash the game profiles in such a way that games with high Jaccard similarity end up in the same bucket.

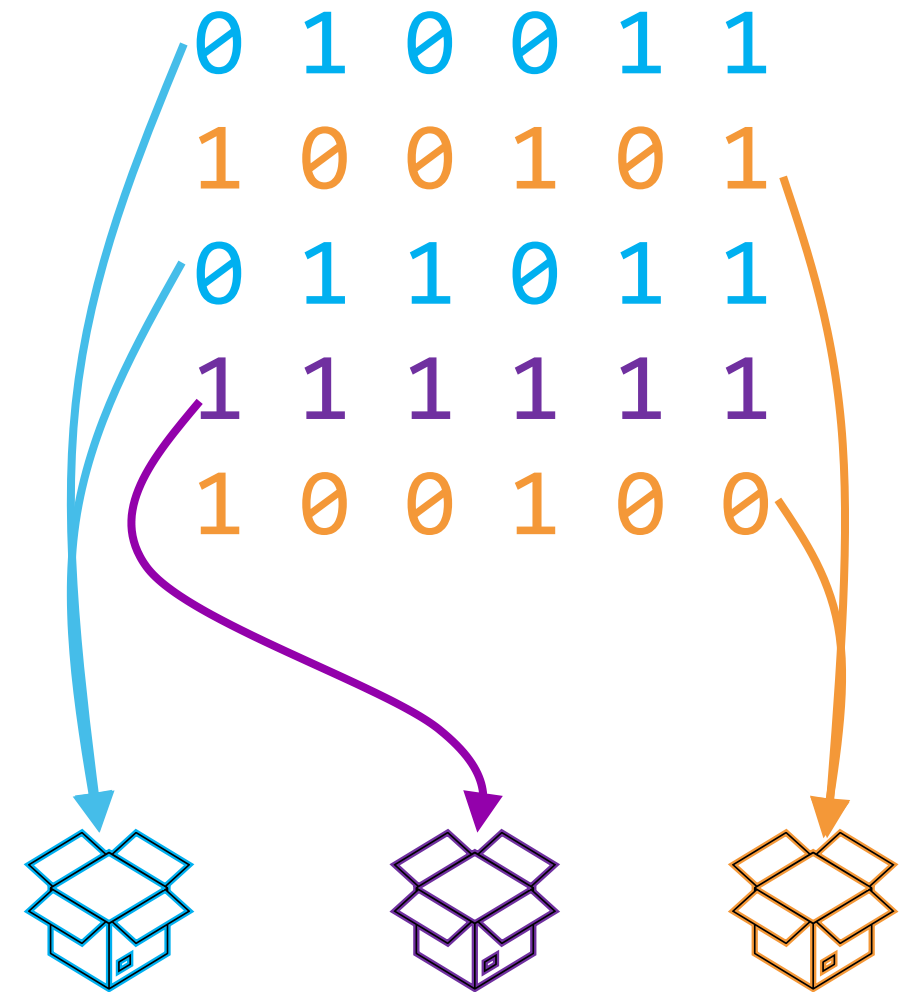
0	1	0	0	1	1
1	0	0	1	0	1
0	1	1	0	1	1
1	1	1	1	1	1
1	0	0	1	0	0



A SIMILARITY-PRESERVING HASH FUNCTION

The key insight is that in a standard hash table, collisions are a bad thing. For this problem, collisions are a good thing – what we would like to do is hash the game profiles in such a way that games with high Jaccard similarity end up in the same bucket.

What hash function might we use for this purpose?



MINHASHING

Cool idea: Take the columns of the profile and permute them randomly. Then count the number of leading zeros and use that as the value of the hash function.

Observe that rows 2 & 5 land in the same bucket as desired, as do rows 1 & 3. Row 4 ends up in the same bucket as 1 & 3 – not ideal, but its similarity with row 3 is actually 4/6, so this isn't too bad.

More generally, what is the probability that two rows land in the same bucket?

0	1	0	0	1	1
1	0	0	1	0	1
0	1	1	0	1	1
1	1	1	1	1	1
1	0	0	1	0	0
1	0	0	1	0	0

1	0	0	0	1	1	= 0
0	0	1	1	1	0	= 2
1	1	0	0	1	1	= 0
1	1	1	1	1	1	= 0
0	0	1	1	0	0	= 2

ANALYSIS OF MINHASHING

Without diving into the details (because it's a homework problem), the probability that two profiles hash into the same bucket using minhashing is precisely equal to their Jaccard similarity!

This means that when we apply minhashing, we generate buckets of items that are likely to be similar. And we can do this in linear time, because we avoid pairwise comparisons.

If we want more reliability, we can stack the deck by performing multiple minhashes...

MINHASH SIGNATURES

If we compute two distinct minhashes, the probability for profiles p_1 and p_2 that both of them match is $(J(p_1, p_2))^2$. This greatly decreases the odds that two dissimilar profiles land in the same bucket.

Conversely, if we ask for the probability that at least one of the hashes matches, that probability is $1 - (1 - J(p_1, p_2))^2$. This greatly decreases the odds that two similar profiles don't land in the same bucket.

We can combine both of these approaches...

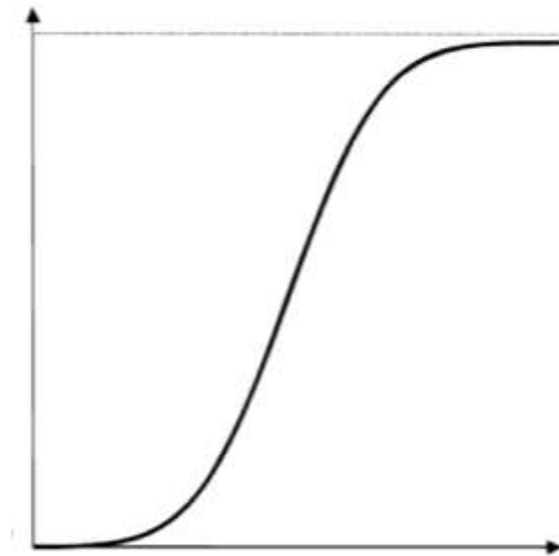
MINHASH SIGNATURES

A typical use of minhashing would be to compute, say, 20 hashes in four groups of five.

Within a group, we force all hashes to match, but then we say that if any of the four groups is a 5/5 match, we put them in the same bucket.

The resulting probability graph is a tunable S-curve, which we can make as steep or shallow as we like, with a threshold Jaccard similarity of any value.

In short, we can make it so the probability curve approximates a step function.



BEFORE YOU LEAVE

Please complete this feedback form about today's class.

